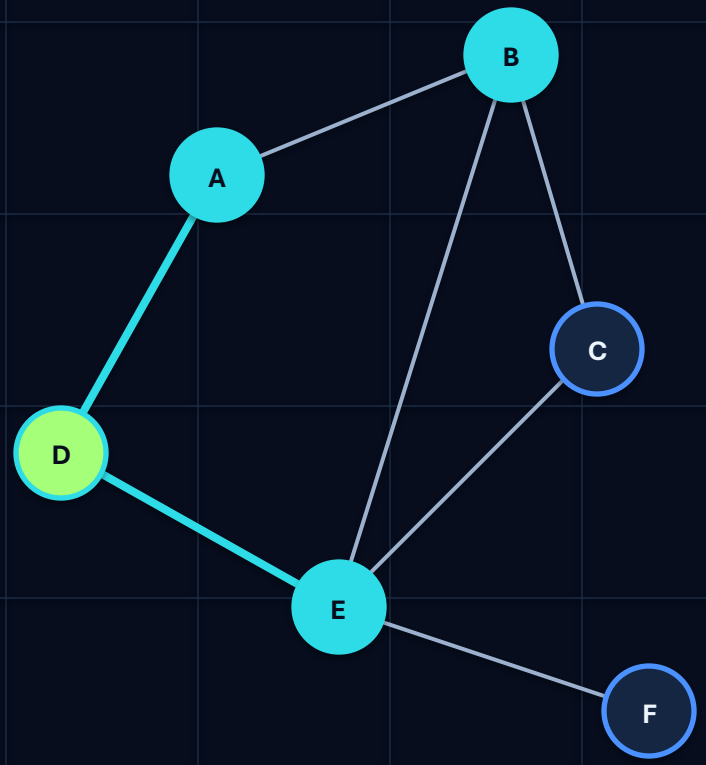
ALGORITHM EXPLORATION DIVISION / DOSSIER 001

DFS MISSION CONTROL

Depth-First Search, visualized as a live exploration mission.



MISSION OBJECTIVE

Explore deep • Track every visit • Backtrack safely

THE COMPLETE DFS MASTERCLASS • THEORY + TRACE + C

What you will learn

A complete route from intuition to implementation.

01

MODEL

Represent a problem as vertices and edges.

02

EXPLORE

Visit one branch deeply before switching.

03

TRACK

Use visited[] and a recursion/explicit stack.

04

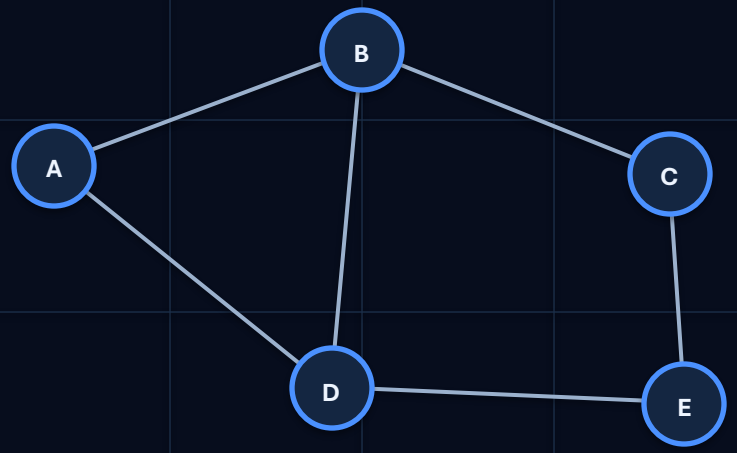
PROVE

Understand correctness and complexity.

By the end: you should be able to trace DFS by hand, explain the stack, write C code, and defend its complexity in a viva.

First, what is a graph?

DFS explores a graph; it does not require a tree.



GRAPH = (V, E)

- V = set of vertices (nodes)
- E = set of edges (connections)
- Undirected edge: A—B works both ways
- Directed edge: A → B has a direction

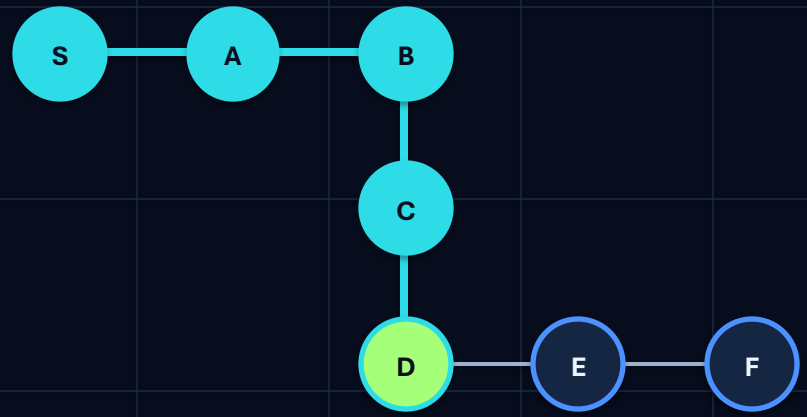
Think of nodes as locations and edges as roads. DFS is the exploration strategy used to navigate them.

DFS: keep going until you cannot

Go deep along one available path; when blocked, backtrack.

THE EXPLORER'S RULE

- Choose a start vertex.
- Mark it visited immediately.
- Choose an unvisited neighbour.
- Repeat until no unvisited neighbour remains.
- Backtrack to the most recent choice point.



The green node is the current frontier.

The DFS algorithm in five moves

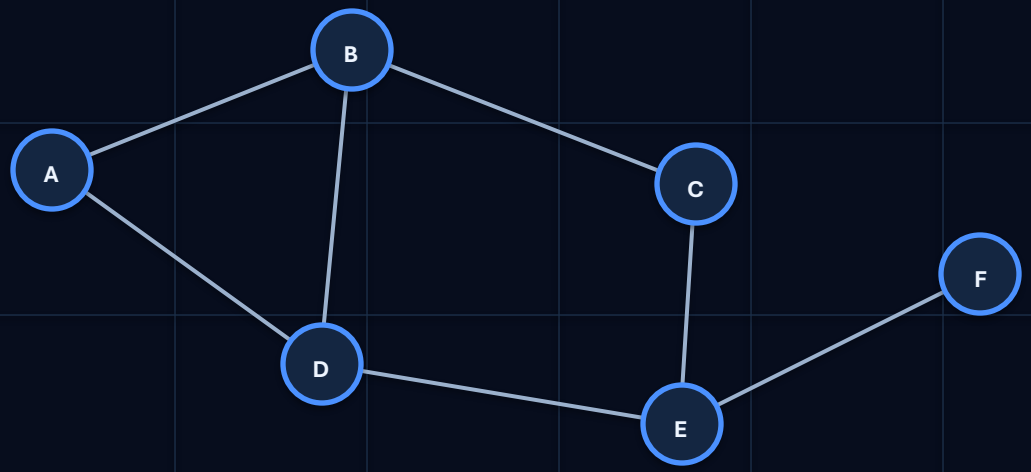
The order of these steps is important.

01	ENTER	Receive a vertex u.
02	MARK	Set visited[u] = true.
03	SCAN	Inspect each neighbour v of u.
04	RECURSE	If v is unvisited, run DFS(v).
05	RETURN	Finish u after all neighbours are handled.

Critical rule: mark a vertex visited before exploring its neighbours.

Our mission graph

Assume neighbours are explored in alphabetical order for this trace.



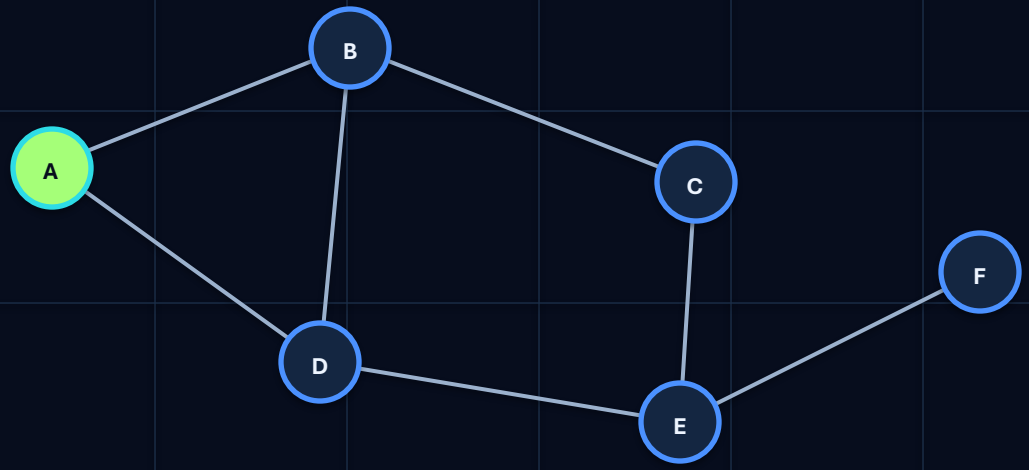
TRACE SETTINGS

- Start = A
- Order = A→B first
- Undirected graph
- Visited nodes stay marked

Traversal can change when neighbour ordering changes. DFS is a strategy, not one universal visit order.

Step 1 — enter A

Mark A before scanning its neighbours.



STACK / CALLS

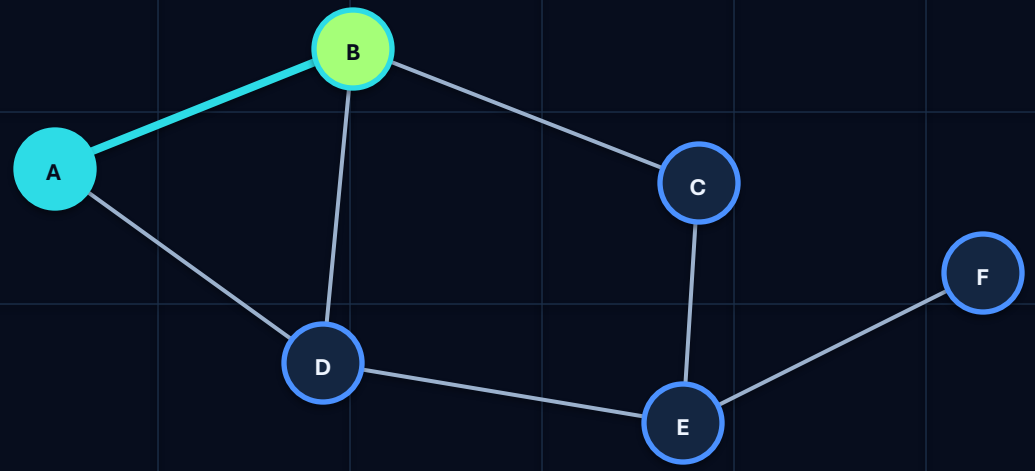
[A]

Visited: {A}

Next unvisited neighbour of A: B (alphabetical order).

Step 2 — go deeper to B

DFS does not finish A yet; A remains on the call stack.



STACK / CALLS

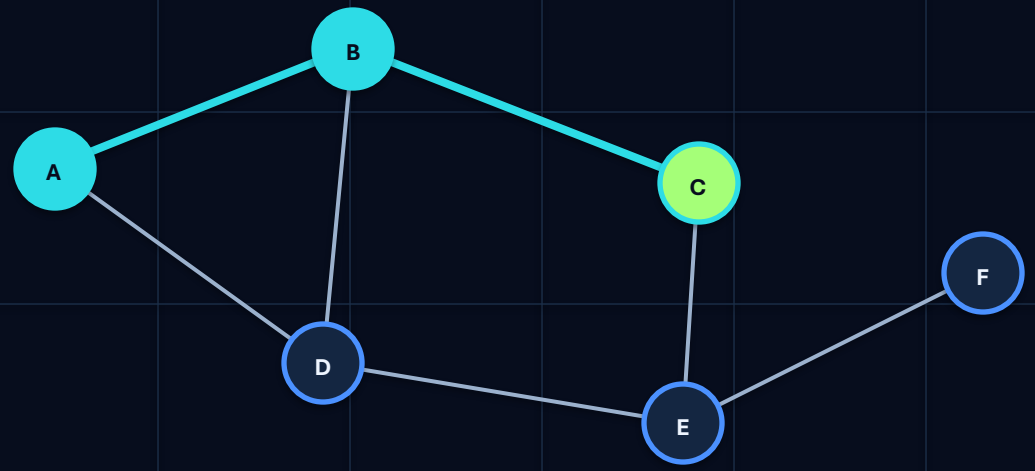
[A]
[B]

Visited: {A, B}

B's next unvisited neighbour is C.

Step 3 — descend to C

The active path now reaches the far side of the graph.



STACK / CALLS

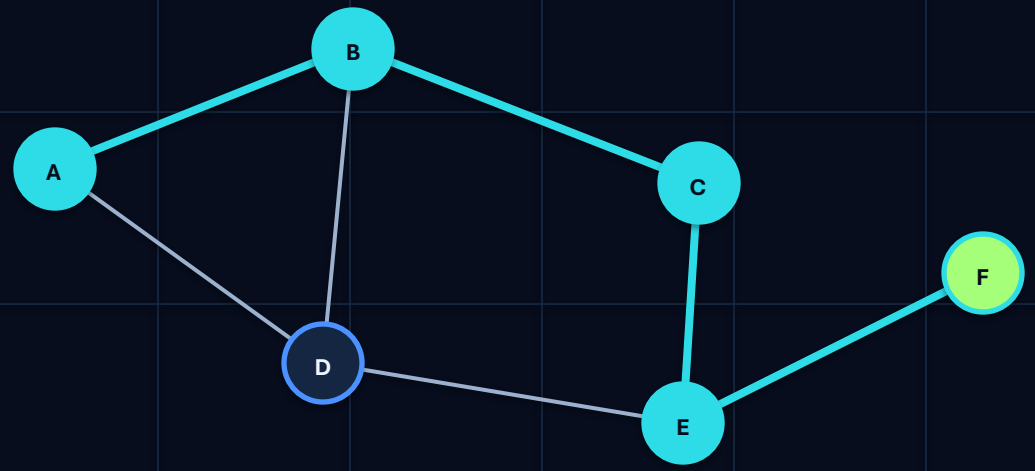
[A]
[B]
[C]

Visited: {A, B, C}

C's neighbour E is unvisited, so DFS continues to E.

Step 4 — reach E, then F

DFS keeps descending while a fresh neighbour exists.



STACK / CALLS

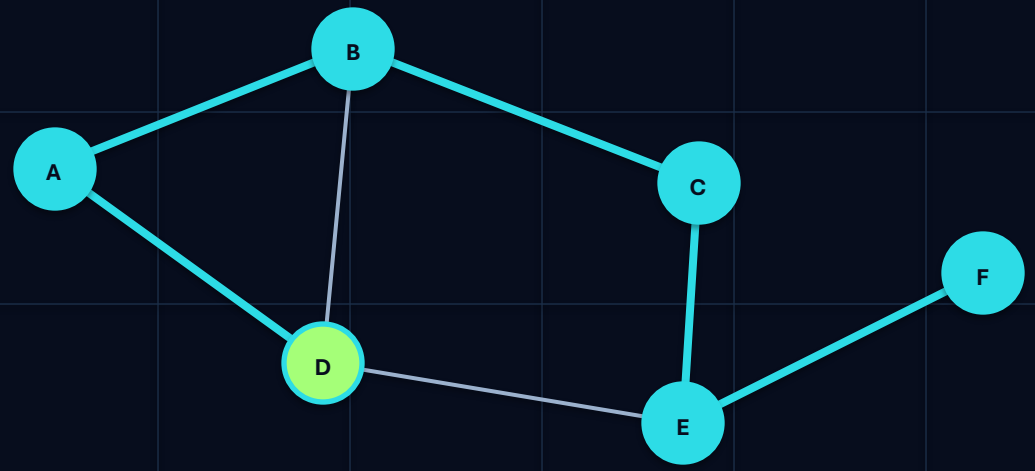
[A]
[B]
[C]
[E]
[F]

Visited: A,B,C,E,F

F has no unvisited neighbour. The algorithm begins to backtrack.

Step 5 — backtrack to D

Returning from F and E exposes the remaining unvisited branch D.



STACK / CALLS

```
... return F
... return E
... return C
... return B
```

All vertices visited

Traversal order: A → B → C → E → F → D

Traversal log — mission complete

With alphabetical neighbour order, the DFS discovery order is:



DISCOVERY ORDER ≠ EDGE LIST ≠ STACK ORDER

Discovery order records when nodes are first visited; the call stack records unfinished recursive calls.

Why the stack matters

Recursion uses the program call stack; iterative DFS uses an explicit stack.

RECURSIVE CALL STACK

```
DFS(A)  ← waiting for child
DFS(B)  ← waiting for child
DFS(C)  ← waiting for child
DFS(E)  ← waiting for child
DFS(F)  ← current call
```

STACK LIFO BEHAVIOUR

- Push: add a new work item.
- Pop: process the most recent item.
- LIFO = Last In, First Out.
- Backtracking resumes the previous choice point.

Stack is not the same as visited[]: stack tracks active work; visited[] prevents repeated exploration.

Recursive DFS — pseudocode

Language-independent logic; adjacency representation determines how neighbours are scanned.

```
DFS(u):
    visited[u] = true
    output(u)

    for each neighbour v of u:
        if visited[v] == false:
            DFS(v)

# Start one component:
DFS(start)
```

INVARIANTS

- Mark before recursion.
- Only recurse into unvisited neighbours.
- A call returns after scanning all neighbours.
- For all components, launch DFS from each still-unvisited vertex.

C implementation — adjacency matrix

This example is self-contained for the six-node mission graph.

```c
#include <stdio.h>
#define N 6
int graph[N][N] = {
 // A B C D E F
 {0,1,0,1,0,0}, // A
 {1,0,1,1,0,0}, // B
 {0,1,0,0,1,0}, // C
 {1,1,0,0,1,0}, // D
 {0,0,1,1,0,1}, // E
 {0,0,0,0,1,0}  // F
};
int visited[N] = {0};
void dfs(int u) {
    visited[u] = 1;
    printf("%c ", 'A' + u);
    for (int v = 0; v < N; v++)
        if (graph[u][v] && !visited[v])
            dfs(v);
}
int main(void) {
    dfs(0);
    return 0;
```

EXPECTED OUTPUT

```
A B C E F D
```

The order follows the matrix's neighbour scan from index 0 to 5.

```
Compile: gcc dfs.c -o dfs
```

Iterative DFS — explicit stack

Useful when you want to control the stack yourself.

```
push(start)
while stack is not empty:
    u = pop()
    if visited[u]:
        continue
    visited[u] = true
    output(u)

    for each neighbour v of u:
        if not visited[v]:
            push(v)
```

IMPLEMENTATION NOTE

- Push neighbours in reverse order if you want the same alphabetical visitation tendency as recursive DFS.
- Depending on when visited is marked, duplicate entries may be pushed.
- A stack can replace recursion but the traversal order may differ.

How the graph is stored changes the cost

DFS logic is the same; neighbour lookup is not.

ADJACENCY LIST

```
A: B, D
B: A, C, D
C: B, E
D: A, B, E
E: C, D, F
F: E
```

Space: O(V + E)

ADJACENCY MATRIX

```
V × V table
matrix[u][v] = 1 if edge exists
```

Space: O(V^2)

For adjacency lists, DFS runs in O(V + E). With a matrix, scanning all V columns per vertex makes it O(V^2).

Time and space complexity

Always state the graph representation and what the space count includes.

ADJACENCY LIST

$O(V + E)$

Each vertex and edge is processed a constant number of times.

ADJACENCY MATRIX

$O(V^2)$

Each visited vertex scans a row of V entries.

AUXILIARY SPACE

$O(V)$

Visited array plus worst-case recursion/explicit stack.

V = number of vertices • E = number of edges. Total storage of an adjacency list is $O(V + E)$; matrix storage is $O(V^2)$.

Why does DFS find every reachable vertex?

Proof idea: discovery + neighbour scanning + visited invariant.

BASE	The start vertex is marked and processed.
STEP	Whenever u is processed, every neighbour v is checked.
DISCOVERY	If v is unvisited, DFS(v) is called, so v is explored.
TERMINATION	Each vertex is marked once; revisits are skipped.

Conclusion: DFS visits every vertex reachable from the start, assuming the graph representation is correct.

DFS beyond the simple example

The real implementation must account for graph shape and requirements.

- Disconnected graph: one DFS(start) covers only its connected component; loop over all vertices to cover all components.
- Cycles: visited[] prevents infinite revisits.
- Directed graph: follow outgoing edges only; reachability is directional.
- Self-loop: the node is already visited, so do not recurse again.
- Empty graph / invalid start: validate input before calling DFS.
- Large graph: recursive DFS may overflow the call stack; consider iterative DFS.

Production habit: define whether you need one component, all components, or a directed reachability search.

DFS vs BFS

Same graph, different exploration policy.

DFS

- Uses a stack / recursion
- Explores deeply before backtracking
- Useful for cycle detection, components, topological sorting
- Does not guarantee shortest path in an unweighted graph

BFS

- Uses a queue
- Explores level by level
- Finds shortest path in an unweighted graph
- May hold a wide frontier in memory

Memory hook: DFS = Stack + Depth. BFS = Queue + Breadth.

Where DFS is used

DFS is a general exploration primitive, not just an exam algorithm.

01

MAZE SOLVING

Explore corridors and backtrack from dead ends.

02

CYCLE DETECTION

Track active/visited states to detect cycles.

03

TOPOLOGICAL SORT

Use finishing order on a directed acyclic graph.

04

CONNECTED COMPONENTS

Group mutually reachable vertices in an undirected graph.

05

PATH / STATE SEARCH

Explore game states, configuration spaces, and dependencies.

06

FLOOD FILL

Visit connected cells in grids and image regions.

Common mistakes — and fixes

These are the errors most likely to break a viva or implementation.

BUG

- Marking visited after recursion
- Forgetting to initialize visited[]
- Assuming one start covers disconnected graph
- Confusing recursion stack with visited set

FIX

- Mark the node before scanning neighbours
- Initialize all visited entries to false / 0
- Run DFS from each unvisited vertex for a full traversal
- Explain stack = active work; visited = already discovered

Debug question: what happens on a triangle A—B—C—A if visited[] is removed?

Viva rapid-fire + source notes

Practice concise answers; be ready to trace a small graph on the board.

VIVA CHECKPOINTS

- What is DFS, and how does it differ from BFS?
- Why do we need visited[]?
- What is the role of the stack?
- Why is list-based DFS O(V + E)?
- What changes for a disconnected graph?
- Does DFS always find the shortest path? Why not?

REFERENCE NOTES

- Cormen et al., Introduction to Algorithms (graph search).
- Sedgewick & Wayne, Algorithms (graph processing).
- Course notes / DAA lab material for syllabus alignment.

AI disclosure: Generated using ChatGPT (OpenAI).